

YARN in Hadoop

Yet Another Resource Negotiator
Resource Management • Scheduling • Application Execution

Hadoop Cluster Resource Manager

Why Do We Need YARN?

Understanding the limitation of the original Hadoop architecture

Hadoop 1.x

MapReduce was responsible for:

- Resource management
- Job scheduling
- Task execution
- Monitoring

This tightly coupled resource management with one processing model.

The Problem

As clusters and workloads grew, a single framework had to coordinate many resources and jobs.

This reduced flexibility and made it harder to share the cluster with other processing engines.

YARN Approach

Separate the two concerns:

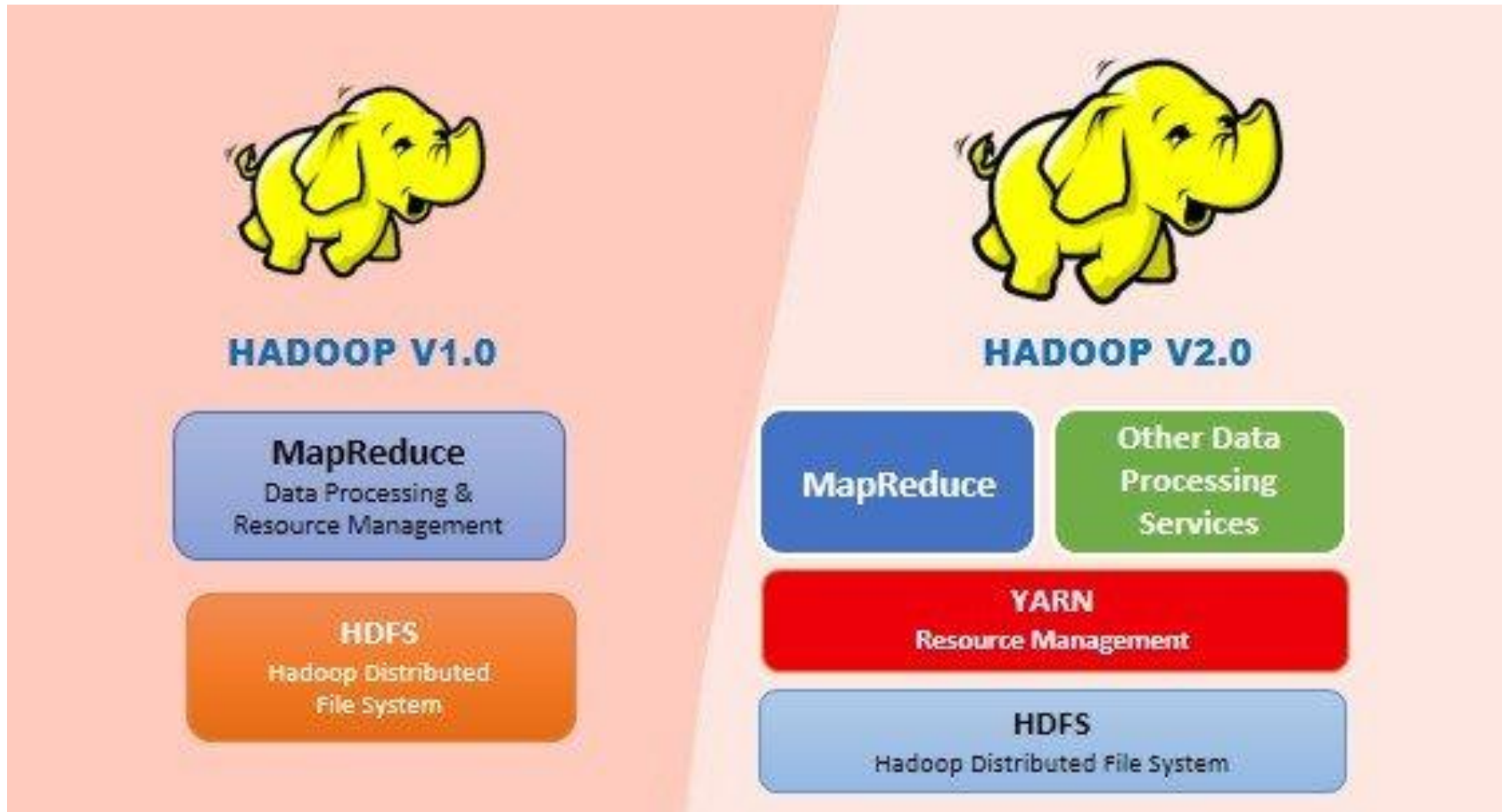
Resource management → YARN

Data processing → MapReduce, Spark and other engines

The cluster becomes a shared computing platform.

Where YARN Fits in Hadoop

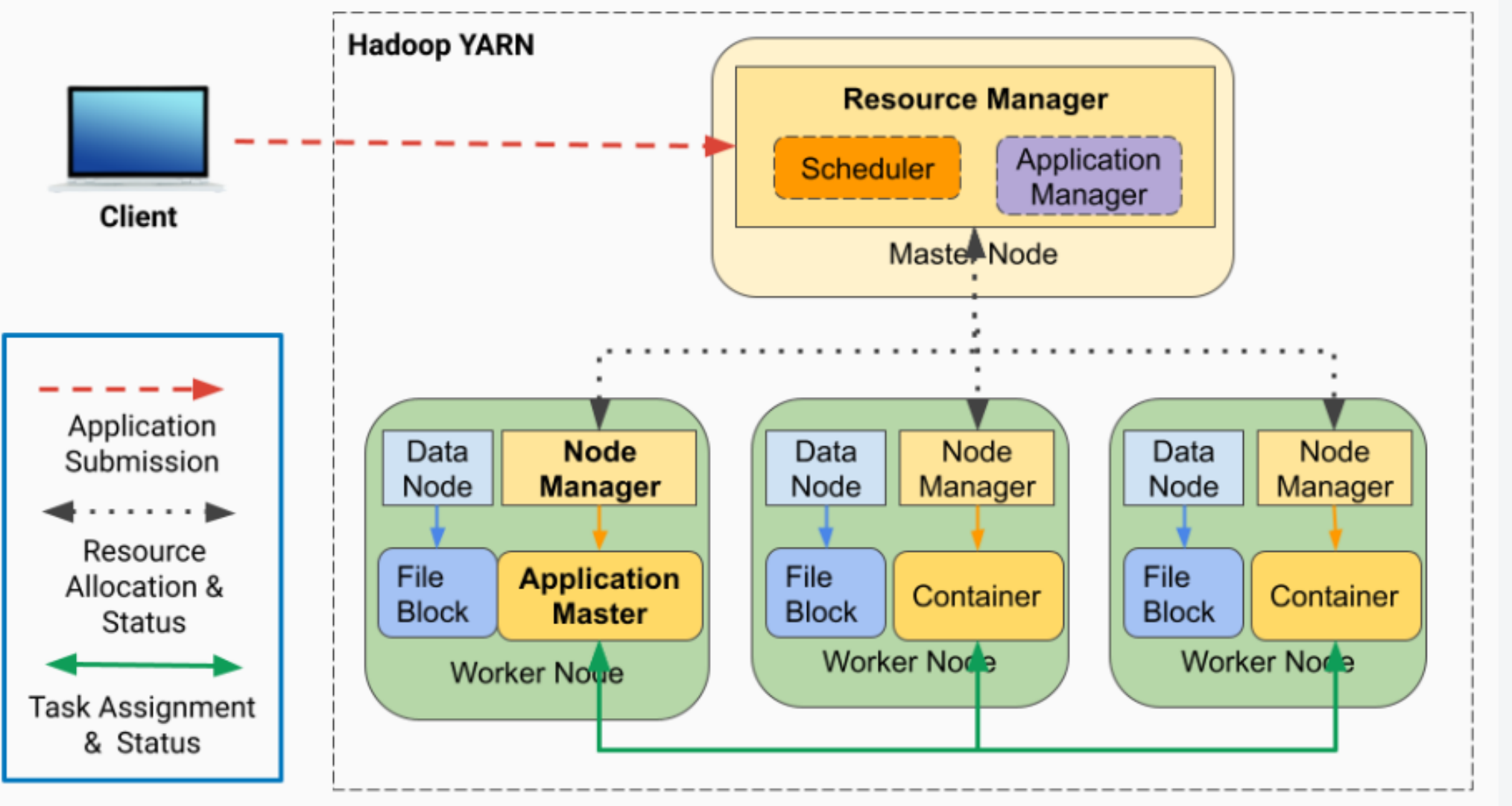
YARN sits between distributed storage and application-level processing



Hadoop 1.x vs Hadoop 2.x with YARN

Aspect	Hadoop 1.x	Hadoop 2.x + YARN
Resource management	Tightly coupled with MapReduce	Handled by YARN
Scheduling	JobTracker	YARN Scheduler
Worker management	TaskTracker	NodeManager
Application control	MapReduce-oriented	ApplicationMaster per application
Processing models	Primarily MapReduce	MapReduce + Spark + others
Scalability	More constrained	Designed for broader cluster sharing

YARN Architecture



ResourceManager

The central authority for resources across the Hadoop cluster

Scheduler

- Allocates resources to applications
- Uses scheduling policies such as capacity/fairness-oriented policies
- Considers queues, priorities and available resources
- Decides where resources can be assigned

ApplicationManager

- Manages application lifecycle
- Accepts application submissions
- Helps start the ApplicationMaster
- Tracks application state and recovery-related information

Cluster View

- Knows available node resources
- Receives status from NodeManagers
- Coordinates resource allocation across the cluster
- Does not execute application tasks itself

NodeManager

The local resource manager on each worker node

Runs as an agent on every worker node in the cluster.

Reports node health, resource availability and container information to the ResourceManager.

Launches containers assigned to the node.

Monitors container resource usage and application processes.

Helps enforce resource limits so one workload does not consume uncontrolled resources.

Provides the local execution environment for application tasks.

ApplicationMaster

The coordinator for a single application or job

Application-Specific

Each submitted application gets its own ApplicationMaster.

It understands the application's execution requirements and coordinates its tasks.

Resource Negotiation

Requests suitable containers from the ResourceManager.

The ResourceManager grants resources according to scheduling policy and availability.

Execution & Monitoring

Works with NodeManagers to launch tasks, monitors progress, detects failures and coordinates completion.

YARN Containers

A container is a bounded allocation of resources on a node

Container = CPU + Memory + Other Resources

Worker Node 1

Container A

Container B

Container C

Container D

Worker Node 2

Container A

Container B

Container C

Container D

Worker Node 3

Container A

Container B

Container C

Container D

Containers provide controlled execution slots. Tasks run inside containers, while NodeManagers monitor them locally.

How a YARN Application Runs

End-to-end flow from submission to completion

1. Client submits

Application is submitted to ResourceManager.

2. AM starts

ResourceManager allocates resources for the ApplicationMaster.

3. AM requests

ApplicationMaster requests containers for application tasks.

4. Containers allocated

Scheduler grants resources according to cluster policy.

5. Tasks launch

NodeManagers start containers and application tasks.

6. Monitor & finish

AM monitors progress; resources are released after completion.

How YARN Schedules Resources

Scheduling determines which applications receive available cluster resources

Input

Applications request resources.

Scheduler considers:

- Available CPU/memory
- Queue/capacity rules
- Priorities
- Fairness and workload requirements

Decision

The ResourceManager's Scheduler allocates available resources.

It does not run tasks itself; it decides resource placement and allocation.

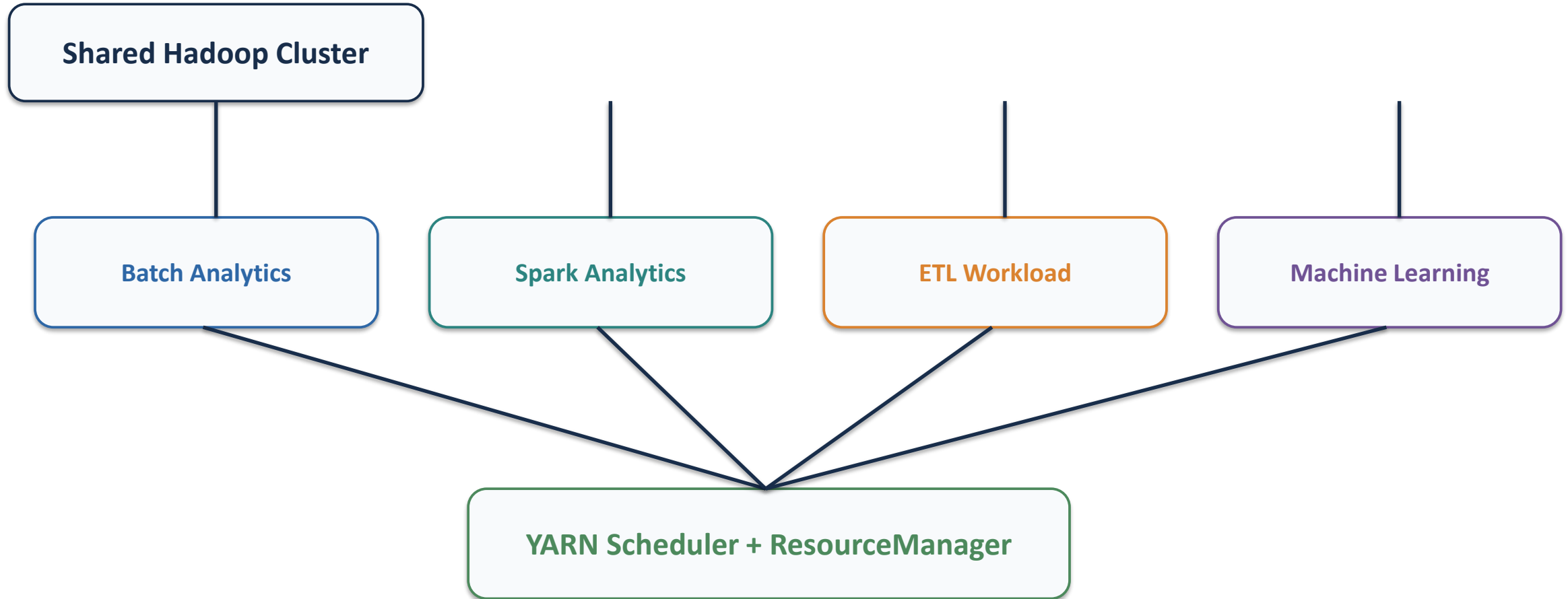
Execution

Allocated resources become containers on worker nodes.

NodeManagers launch and monitor the containers.

Multi-Tenancy and Cluster Sharing

One cluster can support different applications at the same time



YARN enables multiple workloads to share infrastructure while keeping resource allocation centrally coordinated.

Key Benefits of YARN

Scalability

Supports scalable cluster resource management and concurrent applications.

Better Utilization

Shares CPU and memory across applications.

Flexibility

Allows different processing engines to use the same cluster.

Multi-Tenancy

Multiple users/applications can share infrastructure.

Separation of Concerns

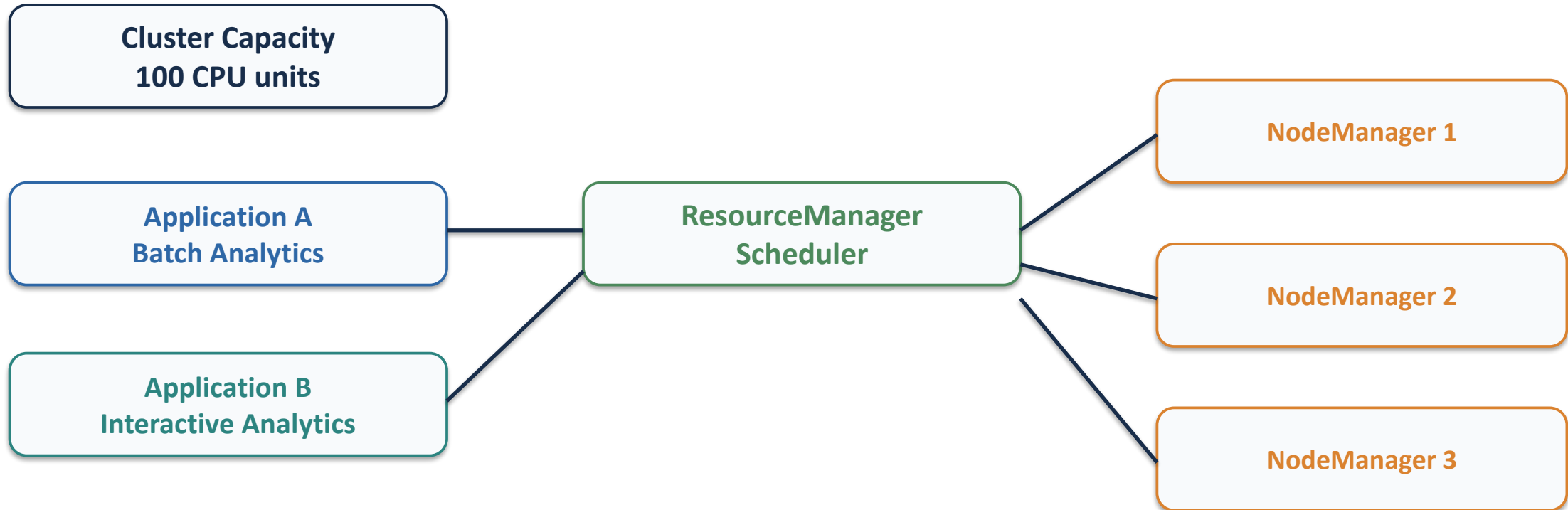
Resource management is independent of processing logic.

Central Scheduling

Cluster-wide allocation is coordinated through the ResourceManager.

Illustrative Example

Two applications compete for the same cluster resources



The Scheduler allocates available resources according to configured policies, queues and resource requirements. Tasks then run inside containers managed by NodeManagers.

YARN with Common Processing Frameworks

MapReduce

Batch-oriented distributed processing.

YARN provides the resources and coordinates execution while MapReduce performs the computation.

Apache Spark

General-purpose distributed analytics.

Spark can request cluster resources through YARN and execute tasks in allocated containers.

Hive / SQL Workloads

High-level data processing and analytics can share cluster resources through the YARN layer.

Other Apps

YARN provides a common resource-management layer that can support multiple distributed applications and frameworks.

Scheduling Policies

Attribute	FIFO Scheduler	Capacity Scheduler	Fair Scheduler
Resource Distribution	Sequential allocation; one giant job can block others.	Assigned via fixed organizational queue percentages.	Distributed evenly and dynamically across all active apps.
Setup Complexity	None (Default out-of-the-box configuration).	Moderate; requires configuring capacity limits per queue.	Complex; requires setting up user-placement rules and policies.
Resource Allocation Metrics	Considers request arrival order.	Primarily memory-based but can scale to absolute metrics.	Memory-only by default, or memory + CPU using Dominant Resource Fairness (DRF) .